PARASOFT vs sonarqube

ESL Engineering Software Lab

Static analysis on C code with 50 known defects

Parasoft C/C++test Standard CLI and SonarQube CLI, run against the same ThreadX Win32 source states.

WHAT WAS MEASURED

50 valid planted C defects in six families, mapped line by line to the findings each tool produced. Every verdict in this deck comes from the committed reports, not from a summary.

SOURCE STATES

```
harness-clean  844260e1
bugged  9d699595
52 markers · 50 valid on Win32
```

HEADLINE

Latest Parasoft result: **21 of 50** valid defects with all Flow rules enabled at explicit Thorough depth. MISRA C:2025 added no further strict detections. SonarQube code analysis remains unavailable pending its license.

One source tree, one build, two analyzers

target: threads & size arithmetic
candidates: 52 markers, 50 valid

The question. Given C code with known, deliberately hard defects, do two mainstream tools report the same ones? Each finding was mapped back to the catalog entry it describes, so a verdict is never given for a finding that merely lands near a marker.

Held constant. Same machine, same MSVC v145 Win32 build data, same immutable git tags, same defect catalog. Only the analyzer differed.

Scored. 50 valid candidates after a post-scan audit removed two markers that are not defects on this target.

ADJUDICATION RULES USED FOR EVERY CANDIDATE

TRUE POSITIVE	Finding sits at, or on a path directly implicated by, a catalog site, and its message describes that defect's root cause.
PARTIAL / INDIRECT	Related but not the cataloged defect (a lost resource instead of a double free, or a hint rather than the dereference). Recorded separately, still counted as a miss for strict recall.
STRICT MISS	Valid catalog defect with no root-cause finding. Direct misses plus partial/indirect findings.
NOT SCORED	Findings in pristine upstream ThreadX are real-code findings of that project, not false positives of this evaluation. Compiler warnings are excluded.

01
Plant defects
52 marked candidates added between two signed tags.

02
Build both states
azure_rtos.sln, Release|Win32, MSVC v145 build data.

03
Scan
Parasoft: paired clean/bugged follow-ups, all Flow rules and Flow + MISRA. SonarQube: earlier DEEP requests were rejected.

04
Adjudicate
Every finding mapped to a catalog entry by line and message.

05
Audit the benchmark
Two candidates were invalidated; one real issue found in the "clean" baseline.

Read the SonarQube column carefully. The earlier requests were rejected with HTTP 403 before any code file was analyzed. The code comparison is still pending an active license; a rejected scan is not a zero-finding result.

Parasoft C/C++test Standard — the analyzer that ran

ITEM	VALUE RECORDED IN THE SCAN PROTOCOL
Product	C/C++test **Standard 2026.1.3** · build `10.7.4.20260825B5553`
Interface	Command line — `cpptestcli.exe` (Standard CLI, no IDE)
License	Network license server on the same host; edition `server_compliance_edition`
Host	Windows 11 (10.0.26200) x64, machine id `WIN32-2037c5fc`
Compiler	Visual Studio Professional 2026 (VC toolset **v145**), passed to the tool as `-compiler vc_14_5`
Input	Build data collected from `azure_rtos.sln@Release|Win32`
Output	Native HTML/XML reports per run, normalized findings CSVs, source-linked adjudication matrix and hashes
Isolation	Separate worktree per tag, separate tool workspace per run — no state carries between runs

WHY THIS TOOL IS THE INTERESTING ONE HERE

It performs interprocedural flow analysis and models ThreadX API semantics — byte-pool and block-pool lifecycle, mutex ownership rules, kernel-object create/delete. Several planted defects were designed to be reachable *only* if the analyzer knows what those APIs mean, which is what makes the results on the concurrency and pool families worth reading.

TEST CONFIGURATION USED

Original scan: `builtin://Flow Analysis Aggressive`, 15/50; the preset did not explicitly set Thorough depth. **Latest:** all 144 Flow checkers, explicit Thorough depth (4), 240-second hotspot limit and the same TX API model.

A separate profile added all 388 shipped MISRA C:2025 checkers. It produced more compliance diagnostics but no additional strict planted-defect TP.

All four latest runs exited 0 and checked 214 files; 183–210 Flow-analysis timeouts remain. Native reports and UTC metadata are retained with raw-report hashes.

SonarQube CLI — what it can and cannot execute here

ITEM	VALUE RECORDED IN THE SCAN PROTOCOL
Product	SonarQube CLI **1.9.0** · build `1.9.0.15656`, installed from the official SonarSource install script
Interface	Command line — `sonar.exe`, including `sonar analyze` with `--depth DEEP`
Authentication	Signed in to SonarQube Cloud (EU region), organization `zuwasi`; token held in the OS keychain, status reported healthy
Project	`zuwasi_parasoft-vs-sonarqube-threadx`, imported from the public repository with the owner's approval
Engines invoked	Secrets scanner (`sonar-secrets 2.49.0.12346`) — runs locally; and Vortex DEEP code analysis — runs on Sonar's service
Commands tried	Whole-scope: `--file ×8 --depth DEEP` · change-set: `--base harness-clean --depth DEEP` · `analyze secrets`
Code result	**None** — Vortex returned `403 Forbidden` on every project-backed attempt; all requested files listed as skipped
Secrets result	0 secrets found in each state, on the eight evaluated C files

The one sentence that matters.
`Vortex analysis is not available on this connection (403 Forbidden)`
And, from `sonar system status`: authentication healthy, **VORTEX Status: Not entitled**.

THE TWO ENGINES ARE NOT THE SAME THING

The local secrets scanner found no credentials — that is a real, completed analysis, and it says nothing about C defects. The engine that would have analyzed the 50 planted defects is the service-backed deep code engine, and it never ran.

WHY THE DIFFERENCE MATTERS FOR THIS DECK

A zero-secrets result is often quoted as "SonarQube found nothing". It is not a code-analysis result and is never used as a score anywhere in this deck.

ThreadX Win32 sample application plus an evaluation harness

6 files changed by the plants
+219 / −122 lines

HARNESS SOURCE FILES — THE DEFECT SITES LIVE HERE

FILE	LINES	ROLE IN THE PIPELINE
harness_common.h	247	Types, frame/header layout, limits, error codes
harness_init.c	463	Creates threads, queues, pools, mutexes, event group
harness_sizearith.c	306	Allocation sizing, length arithmetic, ring index, header store
harness_memory.c	316	Frame lifetime, pools, batch send, cache, retry
harness_concurrency.c	209	Shared stats, slot hand-off, lock order, job list, queue drain
harness_nullpaths.c	238	Lookup and validation paths, out-parameters, callbacks
harness_taint.c	185	Queue-sourced indices, lengths, names, format strings
harness_uninit.c	176	Pooled memory, stack headers, CRC scratch, snapshots

The scans also cover pristine upstream ThreadX kernel code in the same solution. Its findings are reported separately and never counted as benchmark false positives.

THREADX OBJECTS CREATED BY THE HARNESS

4 threads · 3 queues · 1 byte pool
1 block pool · 4 mutexes · 1 event group
producer · consumer · logger · maintenance

WHERE THE DEFECT INPUTS COME FROM

Trigger data arrives from queue receives (`tx_queue_receive` into caller memory) and from volatile simulated sensor registers. At runtime the producer only writes documented in-range values, so the application keeps running; statically, a cross-thread queue payload is an unknown value and a volatile read is a fresh unknown.

Runtime-benign by design. A 20-second soak ran with all 52 plants present and the process stayed alive. That establishes only that — it is not proof that every defect is harmless, and the deck never treats it as such.

The detection target is a git diff, not a wish list

52 markers planted
50 valid on this target

GIT STATE	TAG	COMMIT	ROLE
Pristine upstream ThreadX	`baseline-clean`	`93387b0a`	Reference for upstream findings
Win32 example build adapted to VS2026 / v145	—	`bb0c450c`	Buildable state
Harness added (intended clean baseline)	`harness-clean`	`844260e1`	Clean state scanned
+ 52 planted candidates	`bugged`	`9d699595`	**Detection target**

Both tools scanned the same two materialized states via `git worktree` checkouts of these immutable tags. For the bugged state the Sonar change-set base was `harness-clean`, so the change set *is* the 52 defect sites.

EVERY CANDIDATE IS MACHINE-MAPPABLE

Each plant carries an in-source banner at the defect line:

```
/* [PLANTED DEFECT BUG-003 | CWE-193 | off-by-one:
   idx == LOG_DEPTH passes the bound check and
   writes one past the ring] */
```

A finding can therefore be tied to its catalog entry by line and message instead of by judgement.

Two markers are not defects on Win32. BUG-005 and BUG-031 survived review as invalid (the arithmetic cannot wrap; the range guards are identical). Scoring uses the 50 valid candidates — the pair is still shown, with its evidence, on slide 24.

`BUG-001…012`
Size arithmetic
12 candidates — the emphasized family for this evaluation.

`BUG-013…022`
Memory lifecycle
10 — use after release, double release, leaks, interior free.

`BUG-023…030`
Concurrency
8 — races, lock inversion, TOCTOU, ownership.

`BUG-031…038`
NULL paths
7 valid — unchecked lookup, malloc, out-params, callbacks.

`BUG-039…046`
Taint / bounds
8 — queue-sourced indices, lengths, format strings.

`BUG-047…052`
Uninitialized
6 — pooled memory, stack headers, CRC, snapshots.

How the defects were built to be hard, and fair

design intent, stated in advance
not tuned after the scan

1 · FLOW-ANALYSIS GRADE

Candidates need interprocedural, path-sensitive or dataflow reasoning — a guard that is not exhaustive, a contract the caller ignores, ownership that crosses a function boundary. Rule-matching alone does not reach most of them.

2 · RUNTIME-BENIGN, STATICALLY REACHABLE

Defect branches depend on error paths that never fire under the harness's runtime conditions (queue full, allocation failure, abort request), so the demo keeps running while every defect stays reachable in the analysis.

3 · NO CONSTANT-PROPAGATION GIVE-AWAYS

Bounds checks that exist use live values, and defect inputs come from volatile registers or cross-thread queue payloads that cannot be folded to a constant.

4 · THREADX REALISM

Defects sit inside a real sensor-telemetry pipeline built from the same primitives real firmware uses. Candidates marked `[TX]` additionally require modeling ThreadX semantics — pool lifecycle, mutex ownership, kernel-object create/delete. These probe depth, not pattern coverage.

5 · SIZE ARITHMETIC EMPHASIZED

12 candidates: truncation, wrap, signed/unsigned conversion, `sizeof(pointer)` misuse, masked-length checks, off-by-one. This is the family the report reads first, and the one the deck shows most code for.

RUNTIME INPUTS, DOCUMENTED PER DEFECT

The catalog states what the producer actually writes for each trigger (for example `g_sensor_count = tx_time_get() & 0xF`, so 0–15). That is how "hard statically, benign at runtime" is verified rather than asserted.

Why this matters when reading the results. A defect that a tool misses here is one that needs flow reasoning, not one that is unreachable or hidden behind a constant. The rule set used to attack the set is on slide 20.

Size arithmetic — all 12 candidates, file and line

emphasized family
11 valid · BUG-005 invalid

ID	CWE	SITE	FUNCTION	PLANTED DEFECT
BUG-001	190 → 787	harness_sizearith.c:26	sa_alloc_copy	count*4 truncated to UINT16 for the allocation, full length for the copy
BUG-002	197	harness_sizearith.c:46	sa_copy_checked	Magnitude check admits negative lengths, which convert to huge size_t
BUG-003	193	harness_sizearith.c:63	sa_log_write	idx <= LOG_DEPTH off-by-one writes one entry past the ring
BUG-004	131	harness_sizearith.c:111	sa_frame_new	memcpy of sizeof(pointer) leaves the frame partially initialized
BUG-005	190	harness_sizearith.c:133	sa_pack	**Invalid** — UINT16 payload plus 16 bytes cannot wrap a 32-bit ULONG
BUG-006	197	harness_sizearith.c:154	sa_store_len	Stored length truncated for the allocation, full msg_len used for the copy
BUG-007	787	harness_sizearith.c:176	sa_table_put	Row checked, column not: negative columns walk below the table
BUG-008	131	harness_sizearith.c:196	sa_cfg_snapshot	sizeof(cfg) of the pointer copied instead of the struct
BUG-009	190 → 690	harness_sizearith.c:231	sa_pool_resize	Wrapped pool arithmetic, then an ignored tx_byte_allocate status reaching memset
BUG-010	170	harness_sizearith.c:252	sa_scan_raw	Full-length fill leaves no terminator; the terminator write and strlen both run past the buffer
BUG-011	704	harness_sizearith.c:276	sa_masked_copy	Length checked after a 16-bit mask, copy uses the full 32-bit length
BUG-012	190/787	harness_sizearith.c:295	sa_hdr_store	Configured pad size never validated against sizeof(FRAME_HDR)

Runtime values keep every one of these benign: the producer writes 0–255 to the length registers and 0–15 to the count register, so no truncation or wrap actually occurs while the pipeline runs. The defects are reachable only for values the analyzer must treat as unknown.

The other five families — 38 valid candidates

FAMILY	VALID	REPRESENTATIVE CANDIDATES AND WHAT MAKES THEM HARD
Memory lifecycle	10	`BUG-013` write after `tx_byte_release` · `016` block released to the byte pool · `017` callee releases and returns success · `021` two cache slots alias one frame, so the flush double-frees
Concurrency	8	`BUG-023` unlocked stats update · `025` lock-order inversion versus `net_update_a` · `030` queue count checked, then received with the status ignored
NULL / validation	7	`BUG-033` unchecked `malloc` · `034` out-parameter used after an ignored error · `037` the guard protects the loop but not the returned value · `038` NULL propagated four calls deep
Taint / bounds	8	`BUG-040` length validated against queue capacity · `042` bound checked against a stale table size · `044` received text used as a format string · `046` nibble indexes a 12-entry handler table
Uninitialized	6	`BUG-047` pooled memory read before initialization · `049` a widely called helper stops setting two fields · `051` fill loop stops one short of the compared entry

WHY THE CONCURRENCY FAMILY IS THE SHARPEST TEST

All four threads are created with `tx_thread_create` inside `harness_init.c`. A tool that does not model ThreadX thread creation cannot see that the functions run concurrently at all, and the eight races become invisible rather than merely hard.

THE [TX] CANDIDATES

Eight candidates additionally require ThreadX API semantics: byte-pool memory is not zeroed, a block-pool block must be released with `tx_block_release`, a mutex put requires ownership, a created `TX_TIMER` must be deleted. They probe RTOS knowledge rather than pattern coverage.

COMPILER-VISIBLE CONTROLS

Two plants (BUG-008, BUG-048) are things MSVC itself warns about — C4700 class defects. They are useful as a floor: any flow analyzer that misses the compiler-visible cases would be reported as such.

BUG-001 · the allocation and the copy disagree

harness_sizearith.c:26

PARASOFT: MISS

```
21 UINT sa_alloc_copy(UINT count)
22 {
23     UINT16 size;
24     UCHAR *p;
25
26     /* [PLANTED DEFECT BUG-001 | CWE-190/787 | count*4 truncated to
27        UINT16 for the allocation but not for the copy] */
28     size = (UINT16)(count * 4u);
29     p = (UCHAR *)malloc((size_t)size);
30     if (p == TX_NULL)
31         return HR_FAIL;
32
33     memcpy(p, sa_src, (size_t)count * 4u);
34     free(p);
35     return HR_OK;
```

Trigger: `producer_entry → harness_tick → sa_alloc_copy(count = g_sensor_count)`, a volatile simulated register.

WHAT IS WRONG

Both expressions come from the same value through different routes. For any `count` above 16383 the truncated 16-bit size is smaller than the copy, so the heap buffer is overrun. Detection needs integer-range reasoning that a 32-bit input can exceed that bound, then correlation of the two expressions.

PARASOFT VERDICT — STRICT MISS

No finding on either line, and no finding anywhere in this function. The scan reported two unrelated diagnostics in this file — one on the unused copy in `sa_copy_checked` and one on the ring index at line 68.

SONARQUBE ON THIS DEFECT

NO CODE SCAN The file was among the eight requested; the request was rejected with 403 before analysis. Not a miss — an absence of any result.

BUG-003 · off-by-one, and the finding that proves it

harness_sizearith.c:63

PARASOFT: REPORTED

```
59void sa_log_write(UINT16 code)
60{
61    UINT idx;
62
63    /* [PLANTED DEFECT BUG-003 | CWE-193 | off-by-one: == LOG_DEPTH
64       passes the bound check and writes one past the ring] */
65    idx = (UINT)code >> 3;
66    if (idx <= (UINT)LOG_DEPTH)
67    {
68        log_ring[idx].code  = code;
69        log_ring[idx].seq   = (UINT16)(code ^ 0x5Au);
70        log_ring[idx].stamp = tx_time_get();
71    }
72}
```

The subtlety: LOG_DEPTH is 32 and the ring holds 32 entries, so the correct guard is <. With <=, index 32 passes and writes past the end of the array.

PARASOFT FINDING — FLOW ANALYSIS

```
68BD-PB-ARRAY
 Possibly accessing array
 "log_ring" out of bounds at
 index [0...32]. Correct
 index(es): [0...31]
```

The message names the exact bound the plant violates, which is why this counts as a root-cause true positive rather than a nearby hint.

WHY THIS ONE IS REACHABLE AT ALL

The index is derived (`code >> 3`) from a volatile value, so the analyzer must reason about the derived range against a macro bound instead of folding a constant.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-006 · what the plant removed, on the same function

harness_sizearith.c:154

PARASOFT: MISS

BUGGED STATE — TAG BUGGED

```
149UINT sa_store_len(ULONG msg_len)
150{
151    UINT16 stored;
152    UCHAR *p;
153
154    /* [PLANTED DEFECT BUG-006 | CWE-197 | stored length
155       truncated for the allocation, full msg_len used
156       for the copy] */
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
162    free(p);
163    return HR_OK;
164}
```

CLEAN STATE — TAG HARNESS-CLEAN, SAME FUNCTION

```
152    /* The UINT16 stored length must represent
153       msg_len exactly. */
153    if (msg_len > 0xFFFFu)
154        return HR_FAIL;
155
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored + 1u);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
```

THE FINDING THAT MOVED

On the clean state Parasoft reported `BD-PB-OVERFRD` at line 161 — reading up to 65535 bytes from the 128-byte `sa_src`. In the bugged state the guard and the `+ 1u` are gone and the copy is worse, but the function produced no finding at all. The plant turned a reported condition into a strict miss.

STRICT MISS **BUG-006 scored as a false negative.** An allocation sized from a truncated value while the copy uses the full value is exactly the class of defect this evaluation exists to measure, and the scan did not report it.

BUG-011 · a check that hides the value it should bound

harness_sizearith.c:276

PARASOFT: MISS

```
272UINT sa_masked_copy(ULONG msg_len)
273{
274    UCHAR dst[64];
275
276    /* [PLANTED DEFECT BUG-011 | CWE-704 | length checked after
277       masking to 16 bits, but the copy uses the full
278       32-bit length] */
278    if ((msg_len & 0xFFFFu) < 64u)
279        memcpy(dst, sa_src, (size_t)msg_len);
280    return HR_OK;
281}
```

Concrete counterexample: `msg_len = 0x1003F` (65599). The masked value is 63, so the check passes; the copy then moves 65599 bytes into a 64-byte stack buffer.

PARASOFT VERDICT — STRICT MISS

The scan reports `BD-PB-MEMOPT` at line 279 — "call to memcpy may be optimized out as the dst buffer is not used later". That diagnostic is about the unused copy, not about the length mismatch, so it is not credited as a detection of this defect.

WHY THE DISTINCTION IS KEPT

A finding that lands on the defect line but describes a different problem is exactly how "detected" counts get inflated. Here it is disclosed and scored as a miss.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-018 · the loop that keeps using the freed block

harness_memory.c:175

PARASOFT: REPORTED

```
163UINT batch_send(FRAME *batch, UINT count)
164{
165    UINT i;
166
167    if (batch == TX_NULL)
168        return HR_FAIL;
169
170    for (i = 0; i < count; i++)
171    {
172        if (frame_send(&batch[i]) != HR_OK)
173        {
174            free(batch);   /* whole contiguous block */
175            /* [PLANTED DEFECT BUG-018 | CWE-416 | the loop keeps
176               sending from the freed batch] */
177        }
178    }
179
180    free(batch);
181    return HR_OK;
182}
```

PARASOFT FINDINGS — FLOW ANALYSIS

```
172BD-RES-FREE Usage of freed
 resource "batch"
180BD-RES-FREE Usage of freed
 resource "batch"
```

Two findings, at the failing send and at the unconditional free — the loop-carried use-after-free and the second free it can cause.

WHY THIS COUNTS AS A CLEAN DETECTION

The analyzer tracked the block across `free()` and reported both later uses — the failing send inside the loop and the unconditional free below it. Nothing in the message needs interpretation: it names the object that was released.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-023 · the locked function is right above it

harness_concurrency.c:39

LATEST FLOW PROFILE: TP

```
24void stats_record(ULONG v)          /* correct sibling */
25{
26    tx_mutex_get(&stats_mutex, TX_WAIT_FOREVER);
27    g_stats.total = g_stats.total + v;
28    tx_mutex_put(&stats_mutex);
29}
30
35void stats_record_fast(ULONG v)
36{
37    /* [PLANTED DEFECT BUG-023 | CWE-362 | read-modify-write of
38       shared stats without holding stats_mutex] */
39    g_stats.total = g_stats.total + v;
40}
```

Callers: the producer thread calls `stats_record_fast` while the consumer thread updates the same field through the locked `stats_record`. The two functions are adjacent, in one file, on one shared field.

NEW FINDING — BD-TRS-MLOCK

"The `g_stats` variable is used when locks are not held on `stats_mutex`." At line 39 the read-modify-write omits the mutex used by its sibling function. This is the newly counted BUG-023 TP.

The clean state also reports an unlocked `g_stats` callback at `harness_init.c:65`; that is a separate, unplanted baseline concern and is disclosed in the clean review.

WHY IT IS A FAIR TEST

This latest profile enabled every installed Flow checker, including `BD-TRS-MLOCK`, and used the TX API model. The earlier shipped Aggressive preset missed this site; the full-rule follow-up reports it at the planted update.

SONARQUBE ON THIS DEFECT

NO CODE SCAN 403, file skipped.

BUG-040 and BUG-042 · two bounds errors, both reported

harness_taint.c:71 and :101

PARASOFT: BOTH REPORTED

BUG-040 — LENGTH CHECKED AGAINST THE WRONG CONSTANT

```
66void msg_data_copy(CMD_MSG *m)
67{
68    UCHAR  dst[64];
69    UINT16 len;
70
71    /* [PLANTED DEFECT BUG-040 | CWE-787 | copy length
72       validated against the queue capacity, not the
73       destination buffer] */
73    len = (UINT16)((ULONG)m->param + 4u);
74    if (len <= 128u)      /* dst is 64 bytes */
75        memcpy(dst, m, (size_t)len);
76}
```

```
75BD-PB-OVERFWR Overflow when writing to buffer
 "dst" (UCHAR[64], 64 bytes large). Number of bytes
 written 4 <= len <= 128
```

The message carries the whole argument: the bound in the code is 128, the buffer is 64.

BUG-042 — A CHECK AGAINST A STALE TABLE SIZE

```
95void slot_store(CMD_MSG *m)
96{
97    UINT idx;
98
99    idx = (UINT)m->sensor_id;
100
101    /* [PLANTED DEFECT BUG-042 | CWE-129 | check uses the
102       old table size (8); the slot table has only 4
103       entries] */
103    if (idx < 8u)
104        slot_values[idx] = (ULONG)m->param;
105}
```

```
104BD-PB-ARRAY Possibly accessing array
 "slot_values" out of bounds at index [0...7].
 Correct index(es): [0...3]
```

A bound check *is* present and looks reassuring; the tool compared it with the array's real size.

Both are true positives. This family produced three detections out of eight, all of them array or buffer bounds — the taint path into the value was not the deciding factor.

BUG-049 and BUG-050 · initialization across call boundaries

harness_uninit.c:17 and :127

PARASOFT: BOTH REPORTED

BUG-049 — THE HELPER STOPS SETTING TWO FIELDS

```
15void init_hdr(FRAME_HDR *h)
16{
17    /* [PLANTED DEFECT BUG-049 | CWE-457 | flags and
18       reserved are never initialized by this helper;
19       callers read them] */
20    h->magic       = FRAME_HDR_MAGIC;
21    h->type        = 0;
22    h->seq         = 0;
23    h->payload_len = 0;
24}

 /* five callers build on this helper: frame_build,
    hdr_send, frame_chain_check, batch_send,
    sa_hdr_store. hdr_send reads h.flags. */
```

```
88BD-PB-NOTINIT Non-initialized "h.flags" is used
```

The finding lands at the read site, in a different function from the plant — whole-program initialization tracking.

BUG-050 — AN ERROR CONTRACT THE CALLER IGNORES

```
123UINT slot_track(void)
124{
125    SLOT *u;
126
127    /* [PLANTED DEFECT BUG-050 | CWE-457 | the error
128       contract is ignored: on failure get_slot leaves
129       u untouched] */
129    (void)get_slot(&u);
130    free(u);
131    return HR_OK;
132}
```

```
130BD-RES-INVFREE Invalid memory freeing: "u" is
 not initialized in heap memory
130BD-PB-NOTINIT Non-initialized "u" is used
```

Two diagnostics on one line, both describing the actual defect: the pointer was never assigned because the return value was discarded.

Both are true positives. The Thorough follow-up also detected BUG-048 with BD-PB-NOTINIT, bringing this family's score to 3 of 6.

How Parasoft was run — the original Aggressive scan on both states

original run pair · both exited 0
follow-up profiles on slide 20

```
cpptestcli -config "builtin://Flow Analysis Aggressive" -compiler vc_14_5 \
  -input <worktree>\ports\win32\vs_2019\example_build\azure_rtos.sln@Release|Win32 \
  -report <reports>\parasoft\<state> -workspace <isolated workspace>
```

STATE	CONFIGURATION	UTC START	UTC END	XML FINDINGS	OF WHICH HARNESS
harness-clean	builtin://Flow Analysis Aggressive	18:31:58Z	18:34:05Z	90	6
bugged	builtin://Flow Analysis Aggressive	18:34:06Z	18:36:16Z	**111**	**27**

The clean run is the false-positive baseline and the bugged run is the detection target; both use the same configuration, compiler and build data, so the only difference between them is the planted diff.

WHAT KEEPS THE RUNS COMPARABLE

Each state was materialized as a `git worktree` at an immutable tag, and each run got its own workspace, so no cached analysis leaked from one run into the next. The compiler identifier was resolved once with `-list-compilers` on the first licensed run and reused.

EVIDENCE RETAINED

Two XML reports for this original pair — plus four more from the latest all-Flow and MISRA profiles — with run logs, UTC timestamps, exit codes, normalized CSV exports in the repository, and SHA-256 hashes of every raw report. The raw reports themselves stay outside the repository.

How SonarQube CLI was run — twelve recorded invocations, one working engine

CLI 1.9.0 · org zuwasi · EU region
every invocation, exit code and log is committed

STATE	COMMAND	EXIT	OUTCOME
clean	`analyze --file ×8 --depth DEEP (no project)`	0	No project configured; Vortex skipped
clean	`analyze secrets (whole worktree)`	1	Timed out at the CLI's 30-second limit
clean	`analyze --file ×8 --depth DEEP (no project)`	0	No project configured; Vortex skipped
clean	`analyze secrets (8 C files)`	0	0 secrets
bugged	`analyze --base harness-clean --depth DEEP`	0	No project configured; Vortex skipped
bugged	`analyze --file ×8 --depth DEEP`	0	No project configured; Vortex skipped
bugged	`analyze secrets (8 C files)`	0	0 secrets
clean	`--project … --file ×8 --depth DEEP`	1	**Vortex 403 · 8 files skipped**
bugged	`--project … --base harness-clean --depth DEEP`	1	**Vortex 403 · 6 files skipped**
bugged	`--project … --file ×8 --depth DEEP`	1	**Vortex 403 · 8 files skipped**
both	`sonar system status`	0	Auth healthy · **VORTEX: Not entitled**

SEQUENCE THAT WAS FOLLOWED

Sign in → verify status → import the public repository as a project → attempt whole-file DEEP analysis → attempt change-set analysis against `harness-clean` → narrow the secrets scan to the eight evaluated files after the whole-worktree scan timed out → re-check status. Nothing was skipped in the attempt order; the failures are part of the record.

WHAT THE CLI TOLD THE SCRIPT, VERBATIM

```
Vortex analysis is not available
on this connection (403 Forbidden)
```

```
VORTEX Status: Not entitled
```

The JSON response marks each requested C file as **skipped**, not as clean. Skipped is the word that keeps this experiment honest.

Four controlled scans: all Flow rules, then MISRA

same source tags · compiler · TX model
fresh workspaces · 214 files

HELD CONSTANT ACROSS THE LATEST CLEAN/BUGGED PAIR

Source	`harness-clean` 844260e1 vs bugged 9d699595
Target	`azure_rtos.sln@Release|Win32` · MSVC v145 / `vc_14_5`
Scope	214 files · isolated CLI workspace for every scan
Flow rules	All **144** installed checker IDs; 25 aggressive/unvalidated options plus two NOTINIT options
Depth / time	Thorough (`depthOfAnalysis=4`) · 240-second hotspot limit
RTOS semantics	Same custom ThreadX multithreading API model in both profiles
Second profile	All **388** shipped MISRA C:2025 checkers added; other settings held constant

LATEST PARASOFT FOLLOW-UP · TOTAL / HARNESS FINDINGS

PROFILE	CLEAN	BUGGED	TP	TIMEOUTS C/B
All Flow	324 / 26	354 / 49	**21**	187 / 183
Flow + MISRA C:2025	4,661 / 558	4,745 / 615	**21**	210 / 206

Clean/bugged counts are total XML findings and harness entries. MISRA aliases and compliance diagnostics are not additional planted-defect TPs.

Limit: all runs exited 0, but still reported 183–210 hotspot timeouts. These are completed CLI reports, not exhaustive path analysis. Full commands and native report paths are in `PARASOFT_RUN_GUIDE.md`.

Latest all-Flow profile: 21 of 50 planted defects

42% strict recall · 29 strict misses
50 valid candidates

21 / 50
STRICT TRUE POSITIVES
42% strict recall · explicit Thorough depth

29
STRICT FALSE NEGATIVES
27 direct misses · one partial · one indirect

49
BUGGED HARNESS FINDING ENTRIES
Findings are not unique defects; each TP was adjudicated at its source site.

N/A
SONARQUBE CODE RESULT
Pending license entitlement; prior code requests returned 403

PROFILE	BUGGED TOTAL / HARNESS	STRICT TP	PARTIAL / INDIRECT	STRICT FN
Initial builtin Aggressive (depth not explicit)	111 / 27	15	1 / 1	35
All 144 Flow · Thorough / 240s	354 / 49	**21**	1 / 1	29
All Flow + 388 MISRA C:2025 rules	4,745 / 615	**21**	1 / 1	29

PARTIAL AND INDIRECT SIGNALS REMAIN SEPARATE

BUG-021 (partial): a lost-resource signal, not the planted double free. **BUG-035 (indirect):** BD-PB-CC flags the late null check, not the dereference. Both remain strict misses.

Recall by defect family — where the misses cluster

TP + FN = 50
latest all-Flow Thorough/240s profile

FAMILY	VALID	TRUE POSITIVES	PARTIAL / INDIRECT	DIRECT MISSES	WHAT THE SCAN SAW
Size arithmetic	11	2	0	9	Ring and raw-buffer bounds; truncation, wrap, masked length and sizeof misuse remain missed
Memory lifecycle	10	3	1 partial	6	Use-after-free, interior free and retry leak; no root-cause finding for six other candidates
Concurrency	8	5	0	3	New `BD-TRS-MLOCK` finding for BUG-023; BUG-024, 027 and 030 remain missed
NULL / validation	7	5	1 indirect	1	Unchecked lookup, allocation, out-param and deep NULL propagation
Taint / bounds	8	3	0	5	Buffer overflow and table bounds; indices, format string and allocation size remain missed
Uninitialized	6	3	0	3	Thorough follow-up adds BUG-048; pooled memory and partial-array cases remain missed
Total	**50**	**21**	**1 / 1**	**27 direct**	**29 strict FN**, including partial and indirect

THE HONEST READING OF THIS TABLE

The all-Flow profile adds one race detection without closing the remaining size-arithmetic, memory-contract, or queue/list concurrency gaps. The score is source-adjudicated, not a count of all emitted warnings.

THE EMPHASIZED FAMILY SCORED 2 OF 11

Size arithmetic was the stated focus of this evaluation, and it produced the largest miss block: truncation, wrap, signed-to-unsigned conversion, masked bounds and `sizeof(pointer)` misuse all went unreported.

CONCURRENCY: 5 OF 8

The five root-cause findings require the ThreadX API model and the full Flow checker set. Three planted races/queue defects still have no matching finding.

Parasoft rule messages behind the strict detections

initial detections plus Thorough follow-up
full exports and matrix in the repository

RULE	LOCATION	MESSAGE AS EMITTED BY PARASOFT	CATALOG
BD-PB-ARRAY	sizearith.c:68	Possibly accessing array "log_ring" out of bounds at index [0...32]. Correct index(es): [0...31]	BUG-003
BD-PB-ARRAY	sizearith.c:259	Possibly accessing array "raw" out of bounds at index [2...64]. Correct index(es): [0...63]	BUG-010
BD-RES-FREE	memory.c:172, :180	Usage of freed resource "batch"	BUG-018
BD-RES-INVFREE	memory.c:237	Invalid memory freeing: "body" does not point to the beginning of a resource in memory	BUG-020
BD-RES-LEAKS	memory.c:299, :316	Memory not deallocated: p	BUG-022
BD-PB-NP	nullpaths.c:96	"ch" may possibly be null	BUG-032
BD-PB-CHECKRETGEN BD-PB-NP	nullpaths.c:111, :115	Value returned from method "malloc" should be checked · "s" may possibly be null	BUG-033
BD-PB-NOTINIT	nullpaths.c:154	Non-initialized "f" is used	BUG-034
BD-PB-CC	nullpaths.c:170	Condition "cfg == 0" always evaluates to false	BUG-035 · indirect
BD-PB-NOTINIT	nullpaths.c:211	Non-initialized "v" is used	BUG-037
BD-PB-NP	nullpaths.c:226 (×2)	"c" may possibly be null	BUG-038
BD-PB-NOTINIT	uninit.c:88	Non-initialized "h.flags" is used	BUG-049
BD-PB-NOTINIT BD-RES-INVFREE	uninit.c:130	Non-initialized "u" is used · Invalid memory freeing: "u" is not initialized in heap memory	BUG-050
BD-PB-OVERFWR	taint.c:75	Overflow when writing to buffer "dst" (UCHAR[64], 64 bytes large). Number of bytes written 4 <= len <= 128	BUG-040
BD-PB-ARRAY	taint.c:104	Possibly accessing array "slot_values" out of bounds at index [0...7]. Correct index(es): [0...3]	BUG-042
BD-PB-ARRAY	taint.c:182	Possibly accessing array "handlers" out of bounds at index [0...15]. Correct index(es): [0...11]	BUG-046
BD-RES-LEAKS	memory.c:244, :260	Memory not deallocated: base · Memory not deallocated: ((FRAME_STRUCT[2])frame_cache)[1]	BUG-021 · partial
BD-TRS-MLOCK	concurrency.c:39	The "g_stats" variable is used when locks are not held on "stats_mutex"	BUG-023 · new TP
BD-TRS-ORDER	concurrency.c:75	Possible deadlock - acquiring locks "net_mutex_2", "net_mutex_1" in different orders	BUG-025
BD-TRS-THRDR	concurrency.c:111	The global variable "g_ready" is used without holding a lock	BUG-026
BD-TRS-REVLOCK	concurrency.c:155	The "net_mutex_1" lock is released, but it has not been acquired	BUG-028
BD-TRS-THRDR	concurrency.c:166	The global variable "g_inited" is used without holding a lock	BUG-029
BD-PB-NOTINIT	uninit.c:69	Non-initialized "h.flags" is used	BUG-048

Light-blue rows were absent from the original Aggressive run: BUG-025…029 first appeared with the ThreadX API model at Thorough depth, BUG-048 with the added NOTINIT options, and only BUG-023 needed the full all-Flow profile. MISRA C:2025 repeats the BUG-023 site as MISRAC2025-DIR_5_1-a and is counted once, not as another detection.

What went unreported, and two candidates that are not defects

29 strict misses: 27 direct + 2 related signals
2 of 52 markers invalidated

STRICT MISSES BY CLASS — 29 TOTAL

CLASS	CANDIDATES
Concurrency — 3 direct	BUG-024 · BUG-027 · BUG-030 (BUG-023, 025, 026, 028, 029 detected)
Size arithmetic	BUG-001 truncation · 002 signed/unsigned · 004 sizeof(pointer) · 006 truncated stored length · 007 unchecked column · 008 sizeof(pointer) config · 009 wrapped pool arithmetic · 011 masked length · 012 unvalidated pad
Memory lifecycle	BUG-013 write after release · 014 double release · 015 early-return leak · 016 wrong pool release · 017 release-then-success · 019 timer leak · 021 partial signal
Taint / bounds	BUG-039 unmasked index · 041 strcpy into 16-byte label · 043 unclamped loop · 044 format string · 045 legacy allocation without terminator
Uninitialized	BUG-047 pooled memory · 051 fill loop stops one short · 052 snapshot read before zeroing
NULL paths	BUG-036 unchecked callback · BUG-035 remains an indirect signal

BUG-005 IS NOT A DEFECT ON THIS TARGET

The planted size addition uses a `UINT16` payload, so the maximum total is 65535 + 12 + 4 = 65,551. `ULONG` is 32-bit on Win32, so the addition cannot wrap, and the resulting allocation covers the payload copy. Excluded from valid-defect scoring — and Parasoft correctly reported nothing here.

BUG-031 IS NOT A DEFECT EITHER

The checked accessor and the primary lookup share the same range guard and the same table slot. A non-NULL accessor result therefore proves the primary result is non-NULL. Again, no finding here is the right answer.

Why these are shown rather than deleted. Invalidating the benchmark's own candidates is part of the evidence. Scoring the tools on 52 would have charged them for two misses that do not exist.

The clean reference contains unplanted issues too

all-Flow clean: 26 harness entries
Flow + MISRA clean: 558

GENUINE ISSUE FOUND IN THE CLEAN STATE — HARNESS-CLEAN 844260E1

```
147UINT sa_store_len(ULONG msg_len)
148{
149    UINT16 stored;
150    UCHAR *p;
151
152    /* The UINT16 stored length must represent msg_len exactly. */
153    if (msg_len > 0xFFFFu)
154        return HR_FAIL;
155
156    stored = (UINT16)msg_len;
157    p = (UCHAR *)malloc((size_t)stored + 1u);
158    if (p == TX_NULL)
159        return HR_FAIL;
160
161    memcpy(p, sa_src, (size_t)msg_len);
162    free(p);
163    return HR_OK;
164}
```

```
161BD-PB-OVERFRD Overflow when reading from
 buffer "sa_src" (UCHAR[128], 128 bytes large).
 Number of bytes read 0 <= msg_len <= 65535
```

The allocation is sized `msg_len + 1`, but the 128-byte source can still be over-read for lengths above 128. This issue predates the planted diff and is not credited to any candidate.

OTHER CLEAN-STATE FINDINGS REVIEWED (9 OF 26)

RULE	LOCATION	REVIEW OUTCOME
BD-PB-MEMOPT	sizearith.c:51	Scratch copy with no observable output — intentional
BD-PB-CC	sizearith.c:260	Length already clamped below 64; redundant check
BD-PB-MEMOPT	sizearith.c:277	Scratch copy — intentional
BD-PB-VOVR	memory.c:61	Initial value overwritten before any read
BD-PB-MEMOPT	taint.c:71	Scratch copy — intentional
BD-TRS-MLOCK	init.c:65	Unplanted: clean callback updates g_stats without stats_mutex
BD-TRS-THRDR	init.c:215	ThreadX-managed queue receive, not an app data race
BD-TRS-THRDR	sizearith.c:66–68, 84	File-static ring used only by the logger thread
BD-API-STRSIZE	sizearith.c:161	Duplicate of the known source-buffer over-read

WHAT THIS DOES TO THE COMPARISON

The clean reference already has a real 128-byte source over-read and a separate unlocked callback. They are not planted TPs. Some concurrency warnings are contextual or assumption-dependent; MISRA also emits compliance diagnostics. The 558 MISRA clean entries are not 558 unique defects.

SonarQube code analysis is pending license entitlement

WHAT THE CLI REPORTED, VERBATIM

```
Vortex analysis is not available on this
connection (403 Forbidden)
```

```
SonarQube CLI 1.9.0 system status:
- System check: healthy
- Authentication: active
- Server: https://sonarcloud.io
- Organization: zuwasi
- Vortex: Not entitled
```

Authentication was healthy the whole time. The CLI reached the service, the project existed, the commands were accepted — and the code-analysis engine refused every job.

WHY: AN ENTITLEMENT, NOT A SETTING

Sonar's documentation states that Vortex is part of **Sonar Agent Essentials**, a separate subscription for SonarQube Cloud annual Team or Enterprise plans. The `zuwasi` organization has no Agent Essentials entitlement. No CLI flag, region switch or project setting can grant it, and none was attempted after this was established.

WHAT THIS RESULT IS NOT

NOT ZERO FINDINGS	Every requested file is marked **skipped**. Nothing was analyzed, so nothing was found or missed.
NOT A SONAR SCORE	No TP, FP or FN value for SonarQube exists in this experiment, in either state.
REAL BUT DIFFERENT	The local secrets scanner did complete: 0 secrets in each state, on the same eight files. It does not analyze C defects.

What would change it. An entitled connection — Agent Essentials on this organization, or a supported entitled Server connection. The exact commands in this deck can then be re-run unchanged, and the comparison column becomes a measurement.

Attempt 1
Whole-scope DEEP
8 files requested → 8 skipped, exit 1

Attempt 2
Change-set DEEP
6 changed files → 6 skipped, exit 1

Attempt 3
Status check
Auth healthy · Vortex not entitled

Conclusion
No code result
Every table reports it as pending license, never zero

Measured results, with the profiles kept separate

one harness · one target · one version of each tool
not a product ranking

PROFILE / TOOL	CLEAN TOTAL / HARNESS	BUGGED TOTAL / HARNESS	VALID	TP	PARTIAL / INDIRECT	STRICT FN	STATUS
Initial builtin Aggressive · depth not explicit	90 / 6	111 / 27	50	15	1 / 1	35	Historical baseline
All 144 Flow · Thorough / 240s	324 / 26	354 / 49	50	**21**	1 / 1	29	Completed; timeouts remain
All Flow + 388 MISRA C:2025	4,661 / 558	4,745 / 615	50	**21**	1 / 1	29	No added strict TP
SonarQube CLI · DEEP code analysis	N/A	N/A	50	N/A	N/A	N/A	**Pending license; prior 403**

WHAT THIS EXPERIMENT DOES CLAIM

In the latest all-Flow, ThreadX-aware Thorough scan, Parasoft reported 21 of 50 valid planted root causes and missed 29. The new full-rule scan found BUG-023; MISRA C:2025 aliases that same message and added no strict TP.

WHAT IT DOES NOT CLAIM

This is not a product ranking: one Win32 target, one C/C++test version and TX model, and one entitlement state. All profiles still had Flow timeouts. The SonarQube code result remains **unavailable pending license**, never zero.

HOW TO QUOTE IT SAFELY

Quote the profile and settings: “All installed Flow rules at Thorough/240s found 21/50; adding shipped MISRA C:2025 rules did not increase strict detections.” SonarQube is still pending an entitled code scan.

Everything needed to run this again

PARASOFT C/C++TEST — LATEST COMMITTED PROFILE ON EACH STATE

```
msbuild ports\win32\vs_2019\example_build\azure_rtos.sln \
  /p:Configuration=Release /p:Platform=Win32 /m:1

cpptestcli -config "TX_All Flow Aggressive Thorough 240s.properties" \
  -compiler vc_14_5 \
  -input <state>\...\azure_rtos.sln@Release|Win32 \
  -report <out> -workspace <isolated>
```

The MISRA profile swaps the second committed properties file; `run-threadx-allflow-misra-240.ps1` drives all four scans and `PARASOFT_RUN_GUIDE.md` documents every step.

SONARQUBE CLI — SAME EIGHT FILES, SAME STATES

```
sonar auth login                      # org zuwasi
sonar analyze --file <8 C files> --depth DEEP --format json
sonar analyze --base harness-clean --depth DEEP --format json
sonar analyze secrets <8 C files>
sonar system status                   # shows the Vortex entitlement
```

> **Do not build product code from the bugged tree.** It is an evaluation fork of Eclipse ThreadX with 52 planted candidates.

AUDIT ANCHORS

baseline-clean	93387b0a
harness-clean	844260e1
bugged	9d699595
bugged XML	b1f32ffd81dfc3b0…e96f8
clean XML	235fd61d0eafa584…539c3

Original-pair hashes: `reports/parasoft/SHA256SUMS.txt`. Latest four runs: raw-report and console SHA-256 in `threadx-allflow-misra-240/run-summary.csv`.

COMMITTED EVIDENCE IN THE REPOSITORY

`PLANTED_BUGS.md`	52 candidates, validity audit, trigger paths
`SCAN_PROTOCOL.md` · `PARASOFT_RUN_GUIDE.md`	Versions, commands, UTC times, exit codes, export steps
`…/threadx-allflow-misra-240/`	Latest matrix, four findings CSVs, run summary, clean review
`…/adjudication/` · `…/parasoft/`	Original Aggressive matrix and its findings exports
`…/sonarqube/*`	CLI JSON, secrets logs, outcomes, system status

Every candidate, BUG-001 … BUG-026

TP · FN · PARTIAL / INDIRECT · N/A

Sonar code: no entitled scan (403)

ID	CWE	SITE	ALL-FLOW 240S	SONAR	ADJUDICATION NOTE
BUG-001	190/787	sa:26	FN	N/A	No matching flow finding
BUG-002	197	sa:46	FN	N/A	Only an unrelated unused-copy finding near this routine
BUG-003	193	sa:63	TP	N/A	BD-PB-ARRAY reports ring index 32
BUG-004	131	sa:111	FN	N/A	No matching partial-initialization finding
BUG-005	190	sa:133	INVALID	N/A	UINT16 plus 16 cannot wrap 32-bit ULONG
BUG-006	197	sa:154	FN	N/A	No matching allocation-versus-copy finding
BUG-007	787	sa:176	FN	N/A	No matching unchecked-index finding
BUG-008	131	sa:196	FN	N/A	No matching partial-configuration-copy finding
BUG-009	190/690	sa:231	FN	N/A	No overflow or unchecked ThreadX allocation finding
BUG-010	170	sa:252	TP	N/A	BD-PB-ARRAY reports raw[64]
BUG-011	704	sa:276	FN	N/A	Unused-copy report does not identify the masked-length mismatch
BUG-012	190/787	sa:295	FN	N/A	No matching undersized-header-buffer finding
BUG-013	416	mem:45	FN	N/A	ThreadX release not treated as invalidating the frame
BUG-014	415	con:203	FN	N/A	No finding on the caller's unconditional second release
BUG-015	401	mem:85	FN	N/A	No matching early-return leak finding
BUG-016	762	mem:125	FN	N/A	Wrong ThreadX pool release not reported
BUG-017	415	mem:148	FN	N/A	No finding on the release-on-success contract
BUG-018	416	mem:175	TP	N/A	BD-RES-FREE — use of the freed batch
BUG-019	404	mem:206	FN	N/A	No matching ThreadX timer lifecycle finding
BUG-020	761	mem:233	TP	N/A	BD-RES-INVFREE — freeing an interior pointer
BUG-021	415	mem:257	PARTIAL	N/A	Related lost-resource effect at the alias assignment, not the double free
BUG-022	401	mem:311	TP	N/A	BD-RES-LEAKS — failed-retry buffer leaks
BUG-023	362	con:37	TP	N/A	BD-TRS-MLOCK — unlocked g_stats update (con:39)
BUG-024	366	con:56	FN	N/A	No matching TOCTOU report; init.c:225 leak is a different path
BUG-025	833	con:87	TP	N/A	TX model: BD-TRS-ORDER lock-order deadlock (con:75)
BUG-026	364	con:108	TP	N/A	TX model: BD-TRS-THRDR on unsynchronized g_ready (con:111)

Legend: sa = harness_sizearith.c · mem = harness_memory.c · con = harness_concurrency.c. Verdicts reproduce the committed threadx-allflow-misra-240/DETECTION_MATRIX.csv; the original-run matrix stays in reports/adjudication/.

Every candidate, BUG-027 … BUG-052, and where to check all of it

50 valid candidates · 21 true positives
Sonar code result: unavailable (403)

ID	CWE	SITE	ALL-FLOW 240S	SONAR	ADJUDICATION NOTE
BUG-027	362	con:134	FN	N/A	No matching list race finding
BUG-028	667	con:153	TP	N/A	TX model: BD-TRS-REVLOCK — mutex released unacquired (con:155)
BUG-029	366	con:164	TP	N/A	TX model: BD-TRS-THRDR on unsynchronized g_inited (con:166)
BUG-030	366	con:190	FN	N/A	No queue-status or unchecked-read finding
BUG-031	476	np:74	INVALID	N/A	Same range check means a non-NULL mirror implies a non-NULL primary
BUG-032	476	np:94	TP	N/A	BD-PB-NP — possible null channel dereference
BUG-033	690	np:113	TP	N/A	Allocation return-check and null-dereference findings
BUG-034	457	np:151	TP	N/A	BD-PB-NOTINIT — use of unassigned frame pointer
BUG-035	476	np:166	INDIRECT	N/A	BD-PB-CC flags the late null check as always false; no direct dereference finding
BUG-036	690	np:184	FN	N/A	No matching unchecked-callback finding
BUG-037	476	np:200	TP	N/A	BD-PB-NOTINIT — uninitialized return value
BUG-038	476	np:224	TP	N/A	BD-PB-NP — possible null channel dereference
BUG-039	129	ta:56	FN	N/A	No matching calibration-table bounds finding
BUG-040	787	ta:71	TP	N/A	BD-PB-OVERFWR — destination overflow
BUG-041	120	ta:86	FN	N/A	No matching unbounded-string-copy finding
BUG-042	129	ta:101	TP	N/A	BD-PB-ARRAY — index 4 through 7 in a 4-entry table
BUG-043	787	ta:116	FN	N/A	No matching accumulator-array bounds finding
BUG-044	134	ta:133	FN	N/A	No matching tainted-format-string finding
BUG-045	680/787	ta:151	FN	N/A	No matching allocation-size / string-copy finding
BUG-046	129	ta:179	TP	N/A	BD-PB-ARRAY — handler index 12 through 15
BUG-047	457	un:41	FN	N/A	No finding on pooled frame flags
BUG-048	457	un:63	TP	N/A	BD-PB-NOTINIT — non-initialized h.flags (un:69)
BUG-049	457	un:17	TP	N/A	BD-PB-NOTINIT traces the helper's missing flags initialization
BUG-050	457	un:127	TP	N/A	Uninitialized-use and invalid-free findings at slot_track
BUG-051	457	un:143	FN	N/A	No finding on the uninitialized CRC entry
BUG-052	457	un:167	FN	N/A	No matching uninitialized stats-field finding

Legend: np = harness_nullpaths.c · ta = harness_taint.c · un = harness_uninit.c · mem = harness_memory.c · con = harness_concurrency.c.

Verify any row: PLANTED_BUGS.md · *-findings.csv · latest DETECTION_MATRIX.csv · PARASOFT_RUN_GUIDE.md · run-summary.csv hashes. Repository: github.com/zuwasi/parasoft-vs-sonarqube-threadx (MIT).
Prepared by Daniel Liezrowice, ESL — Engineering Software Lab, 2026-10-08.